المرحلة الأولى — تثبيت مصدر الحقيقة وقواعد الفلترة

الملف المخصص للزميل الأول
الهدف: إصلاح الجذر في schema/SQL وطبقة قراءة الكتالوج وعقدة العرض، دون إعادة تصميم الواجهة.

النطاق

هذه المرحلة مسؤولة عن:

- تعريف العرض المؤهل والعرض الأساسي الأرخص بين متاجر المنصة فقط.
- منع مقارنة الأسعار عبر عملات مختلفة أو توحيد العملة بسياسة موثقة.
- تثبيت ربط الجنس والمناسبات والنوتات بمعرفات/keys مستقرة.
- جعل محرر السيكشن وواجهات الكتالوج تستخدم نفس قواعد السعر والتوفر.
- كتابة اختبارات contract يمكن للمرحلة الثانية الاعتماد عليها.
- إزالة أي اعتماد على صف عشوائي أو خريطة استنتاجية لربط السعر بـ `store_product_id`.
- توحيد سياسة المخزون بين المنصة والمحرر ومراقبة اختلاف النتائج.

لا تعدل في المرحلة الأولى تصميم البطاقات أو تدفق UX للكارت إلا بقدر تغيير شكل البيانات الممررة.

1. عقدة العرض المقترحة

أنشئ مفهومًا واحدًا قابلًا لإعادة الاستخدام، سواء في SQL view/functions أو module JavaScript مشترك:

```
eligibleOffer(product_size_id)
  is_available = true
  sync_status = synced
  store.status = active
  stock_quantity IS NULL OR stock_quantity > 0
  currency policy passes

platformOffer(product_size_id) = minimum eligibleOffer across Élan Scents
storeFilter(product_size_id, store_id) = platformOffer.store_id == store_
```

يجب أن يعاد مع العرض على الأقل:

```
product_size_id, product_id, store_product_id,
store_id, store_name, price, currency,
stock_quantity, price_includes_tax, price_includes_shipping,
last_synced_at, flags
```

أضف tie-breaker نهائيًا ثابتًا بعد السعر/العمولة/الأولوية، مثل `store_id ASC`، حتى لا تتغير النتيجة عشوائيًا عند التعادل.

2. SQL وSupabase

راجع
`supabase/migrations/20261007223643_catalog_data_integrity_and_ranking.sql`
والـ views المرتبطة به:

1. حافظ على `cheapest_offers` كمصدر الأرخص العام لكل `product_size_id`.
2. لا تضف منطقًا يعرض الأرخص داخل متجر كسعر بديل؛ فلتر المتجر يختبر مالك `cheapest_offers` فقط.
3. اجعل شروط الأهلية متطابقة حرفيًا في العرض الأساسي وكل الفلاتر: متجر active، متاح، synced، ومخزون موجب أو NULL.
4. لا تستخدم `products.min_price` إلا كسعر العرض الأساسي بين متاجر المنصة.
5. إن كانت المنصة تعمل بعملة واحدة، أضف constraint/validation يمنع إدخال عملة مختلفة. إن كانت متعددة العملات، أضف سياسة صرف مؤرخة ولا تقارن أرقامًا خامًا.
6. راجع indexes على `store_products(product_size_id, store_id, is_available, sync_status, price)` وعلى علاقات الربط.
7. لا تحذف السجلات التاريخية ولا تغير معنى `NULL stock_quantity`؛ هو مخزون غير معروف وليس صفرًا.

مصدر قراءة موحد

أنشئ View أو RPC موحدًا باسم واضح مثل `public.catalog_eligible_offers` يعيد فقط عروض متاجر المنصة المؤهلة، ويشمل من نفس الصف:

```
store_product_id, product_size_id, product_id,
store_id, store_name, price, currency,
stock_quantity, price_includes_tax, price_includes_shipping,
is_available, sync_status, store_status
```

بعد ذلك يكون `cheapest_offers` أو `catalog_cheapest_offers` هو اختيار `MIN` /الترتيب من هذا المصدر. لا تبنِ الواجهة `storeProductId` من `(store_id, product_size_id)` بعد تحميل السعر من مصدر آخر، ولا تسمح بتكرار الصفوف أو استبدالها بآخر صف مقروء.

3. gender والفلاتر المرجعية

المصدر المقترح:

- `products.gender` : `men` , `women` , `unisex` للفلاتر الثلاثة.
- `occasions` + `product_occasions` : للمناسبات.
- `aromatic_notes` + `product_notes` : للنوتات.
- `product_categories` يبقى للفئات التجارية غير الجنس، أو يُنقل تدريجيًا بعد قياس الاستخدام.

أضف mapping ثابتًا لا يعتمد على ترتيب `texts.js` :

```
men → رجالي / Men
women → نسائي / Women
unisex → للجنسين / A Scent That Unites Us
```

للمناسبات والنوتات، أضف `slug` أو `key` ثابتًا في الجداول، مثل `gift` , `date` , `work` , `evening` , `day` و `cardamom` , `oud` , `woody` , `citrus` , `amber` . لا تجعل الاسم العربي هو المفتاح.

نفذ data audit قبل فرض constraints:

- سجلات بلا gender.
- gender خارج القيم الثلاث.
- أسماء duplicate أو اختلافات `حمضي/حمشي` .
- روابط إلى occasion/note غير نشطة أو غير موجودة.
- منتج مرتبط بقيمة متعارضة بين gender وcategory القديمة.

التنفيذ الجذري للتصنيفات والعلاقات

نفذ migration مستقلة على هذا الترتيب:

1. أضف nullable `slug` / `key` إلى `occasions` , `categories` , و `aromatic_notes` .
2. استخرج تقرير التكرارات والاختلافات الإملائية، ثم املأ keys القياسية: `date` , `work` , `evening` , `daytime` , `gift` و `cardamom` , `citrus` , `woody` , `oud` , `amber` .

3. عالج التكرارات بقرار بيانات واضح، وانقل روابط `*_product` إلى السجل canonical قبل فرض unique constraint.
4. أضف unique constraint على `(key)`، واجعل label `name` قابلًا للتغيير والترجمة وليس مفتاح فلترة.
5. استخدم `products.gender` مباشرة للفلاتر الثلاثة؛ لا تنشئ علاقة category موازية للجنس ولا تجعل ترتيب النصوص مصدر الحقيقة.
6. أنشئ RPC/استعلامًا موحدًا يطبق `EXISTS` لكل بعد: category وoccasion وnote، مع `gender` كقيد على `products`، ثم يعيد `product_size_id` ويربطه بـ `globalOffer`.

القاعدة التنفيذية: AND بين الأبعاد المختلفة، وOR بين عدة قيم داخل البعد نفسه إذا دعم العقد ذلك. استخدم `EXISTS` أو `DISTINCT` لمنع تضاعف الصفوف الناتج عن joins متعددة.

4. عقدة بيانات الواجهة

عدّل loader في `app-core.js` بحيث يحمل:

- `gender` من `public_products`.
- IDs/keys للفلاتر، لا index مرتبة من النصوص.
- كل العروض المؤهلة من `store_products` أو endpoint مخصص للتحقق.
- العرض الأساسي مشتقًا من نفس قاعدة `cheapest_offers`، مع تحقق اتساق في التطوير.

عرّف شكلًا واضحًا لكل perfume size:

```
{
  id: product_size_id,
  pid: product_id,
  sizeMl,
  gender,
  cats: [...stableKeys],
  occs: [...stableKeys],
  notes: [...stableKeys],
  offers: [...allEligibleOffers],
  globalOffer: {...},
  offersByStore: { [storeId]: {...} } // للتحقق فقط، لا لعرض سعر بديل
}
```

لا تستخدم `p.offers` بمعنى مختلف في موضعين. اجعل `globalOffer` صريحًا، وسمِّ المجموعة الكاملة `allEligibleOffers` إذا احتاجت المرحلة الثانية للتحقق؛ لا تسمح لأي helper بإنتاج سعر متجر بديل.

5. editor-catalog

في `railway/editor-catalog.js` :

- أبقِ `size_ml` , `note_id` , `occasion_id` , `category_id` , `store_id` , `brand_id` , `search` , `price_max` , `price_min` .
- اجعل price filters تعتمد على `globalOffer` في كل الحالات؛ وعند وجود `store_id` أضف شرط أن يكون هو مالك `globalOffer` .
- طبّق AND بين الأبعاد المختلفة وOR داخل نفس dimension.
- حافظ على التحقق من UUID والأرقام وحدود pagination.
- وحّد شروط availability مع view العام.
- اجعل نتيجة كل صف تحمل السعر و `store_product_id` من نفس المصدر.
- لا تضف مسارًا يعرض أرخص سعر داخل متجر كبديل؛ `store_id` في المحرر يرشح مالك العرض الأساسي فقط.
- أضف `gender` إلى query ونتيجة الكتالوج، وأعد `note_ids` , `occasion_ids` , `category_ids` كمعرفات/keys لا كفهارس.
- لا تسمح للواجهة بتحويل UUID إلى index؛ labels للعرض فقط.

6. اختبارات إلزامية للمرحلة الأولى

أضف اختبارات SQL/module قبل تسليم المرحلة:

1. عطر واحد، حجم واحد، Store A = 100 وStore B = 120: العام A/100، فلتر A يعيده A/100، فلتر B لا يعيده.
2. Store B غير متاح: لا يظهر في فلتر B، ولا يغير العرض الأساسي إذا كان A مؤهلًا.
3. المخزون NULL: يظهر كمتاح، ولا يظهر إذا كان 0.
4. `sync_status ≠ synced` : لا يظهر.
5. متجر inactive: لا يظهر.
6. price_min/max لا تتغير حسب المتجر المختار؛ تقارن دائمًا سعر العرض الأساسي، والمتجر يرشح مالكه فقط.
7. فلتر gender يعتمد على `products.gender` حتى مع غياب category row.
8. تقاطع note + occasion + gender لا يعيد إلا التقاطع الحقيقي.
9. duplicate names لا تكسر filter key.
10. تعادل السعر يعطي نتيجة ثابتة.
11. currency غير متوافقة تُرفض أو تحوّل حسب السياسة الموثقة.
12. حجم 50ml لا يطابق 100ml لمجرد تطابق product_id.
13. `price` و `store_product_id` يظلان متطابقين بين View، الكتالوج، البطاقة، الكارت، والطلب.

14. عرض `is_available = false` أو `sync_status ≠ synced` أو متجر inactive لا يدخل العروض المؤهلة.
15. نتيجة المحرر والمنصة متطابقة عندما تكون `stock_quantity = NULL` وفق السياسة المعتمدة.
16. تغيير اسم أو ترتيب label لا يغير نتيجة category/occasion/note.
17. `gender` يعمل دون وجود category مقابلة، ولا يستنتج من النص أو اسم العطر.
18. تقاطع ثلاثة أبعاد يستخدم AND، وتعدد القيم داخل البعد الواحد يستخدم OR فقط إذا كان العقد يسمح به.

7. معايير القبول

- لا يوجد أي endpoint أو helper يختار الأرخص داخل متجر ويعرضه كسعر بديل.
- توجد دالة/استعلام واحد موثق للأهلية والاختيار الأساسي بين متاجر المنصة، وشرط واحد لفلتر مالك العرض.
- كل نتائج editor وstorefront في fixtures تعطي نفس product_size IDs والأسعار.
- جميع اختبارات المرحلة الجديدة تمر.
- فحص migrations يمر، ولا توجد migration بلا rollback أو استثناء موثق.
- لا تُنشر migration بيانات قبل استخراج تقرير data audit ومراجعته.
- وثّق قرار مستوى العرض: الخطة الحالية تعرض `product_size_id` كسطر قابل للشراء، ولا تزيل التكرار على مستوى `product_id` في بعض الصفحات دون غيرها.

8. مخرجات التسليم للمرحلة الثانية

سلّم الزميل الثاني:

- contract مكتوب لـ offer context.
- أسماء الحقول النهائية ومثال JSON.
- mapping ثابت للفلاتر.
- fixture يغطي A/B والأسعار والجنس والتقاطع.
- قائمة أي migrations مطلوبة وتوقيت تنفيذها.
- تأكيد أن `store_product_id` هو مفتاح العرض الذي يجب أن يحفظه الكارت عند اختيار متجر.